Building Manager

BuildingManager is the main component of the Advanced Building System. The whole tool is around this script. Support the algorithms and manage the settings.

How does it work?

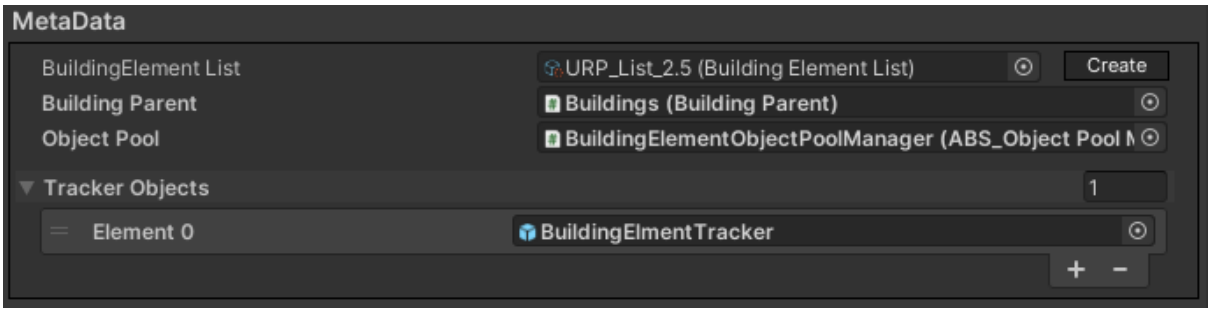
You as a developer should add a BuildingElement to the BuildingManager through the IBuildingManagerExternalInterface. Then the Manager immediately starts to work based on the settings.
Every frame if the BuildingManager has a BuildingElement will make a raycast from the given camera and the manager tries to find the best position to build the BuildingElement.

The BuildingManager can work in 2 different modes. Which can be determined during the activation of the Manager with an enum (BuildingManagerBuildMode).

- Continues
    - The BuildingManager will work until it has been deactivated.
    - If an element has been placed the manager creates a new one and continues the work.
- Single
    - If an element is successfully placed (not more with the drag building at once) the manager will stop working and deactivate itself and doesn't create any new element.

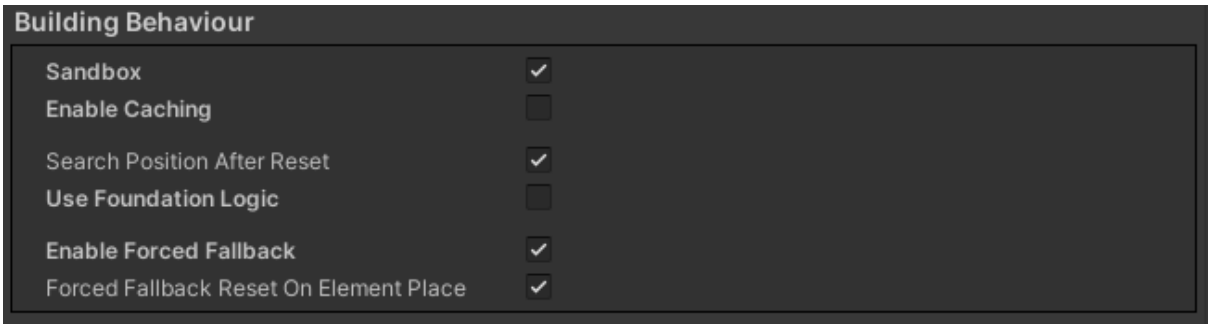
Properties of BuildingManager

MetaData



- BuildingElement List
    - A list of BuildingElements.
    - You can add BuildingElements for the Manager with index if it is contained by this list.
- Building Parent
    - The BuildingParent Object used for creating Buildings.
- ObjectPool
    - ABS_ObjectPoolBase object that will be used to increase the performance of

the BuildingManager.
    - **The Object Pool support feature is explained in detail in the Utility Objects documentation.**
    - This value can be null.
- Tracker Objects
    - A list of the tracker objects that should inherit from the BuildingManagerTracker.
    - With those objects you get the possibility to
        - implement custom functionalities
        - react for custom events
        - make decisions and control the behavior of the BuildingManager.

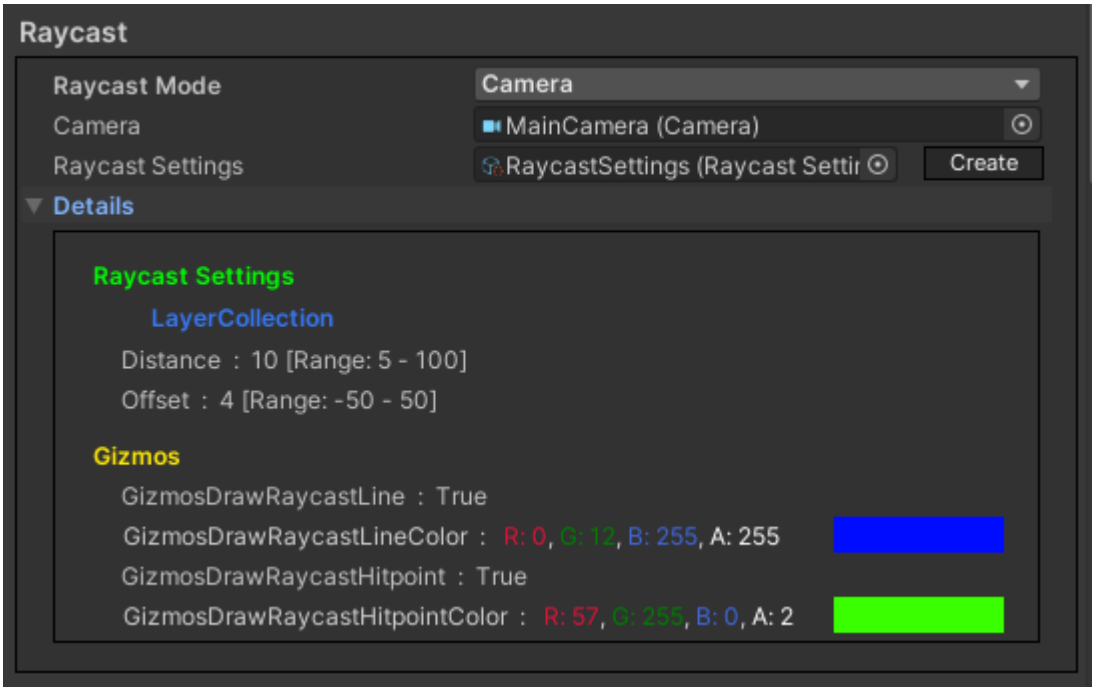


Building Behaviour

- Sandbox
    - It is a boolean.
    - If it is true the BuildingManager will ignore the Maximum Amount of temporary elements so it can place an infinite amount of elements at the same time with DragBuilding.
- Enable Caching
    - If it is true the algorithms will be caching some data to speed up the algorithm's calculations but increase the memory usage of the algorithms.
- Search Position After Reset
    - It is a boolean.
    - If it is true the BuildingManager will perform a position search if the BuildingElement is changed to ensure that if the BuildingElement's Update function is disabled (for example the time is set to 0) the reset of the BuildingElement is fully finished.
- Use Foundation Logic
    - If it is true the BuildignManager allows to build that element by itself what is signed as Foundation.
    - But snapping to an already built element can be done with any BuildingElement.
    - It is only working properly with those algorithms what using snapping logics like AdvancedGrid or SnapPointBased
- Enable Forced Fallback
    - It is a boolean feature that is by default false.
    - If it is true then the Forced Fallback feature is enabled. The Forced Fallback feature will force the usage of the fallback algorithm during the building

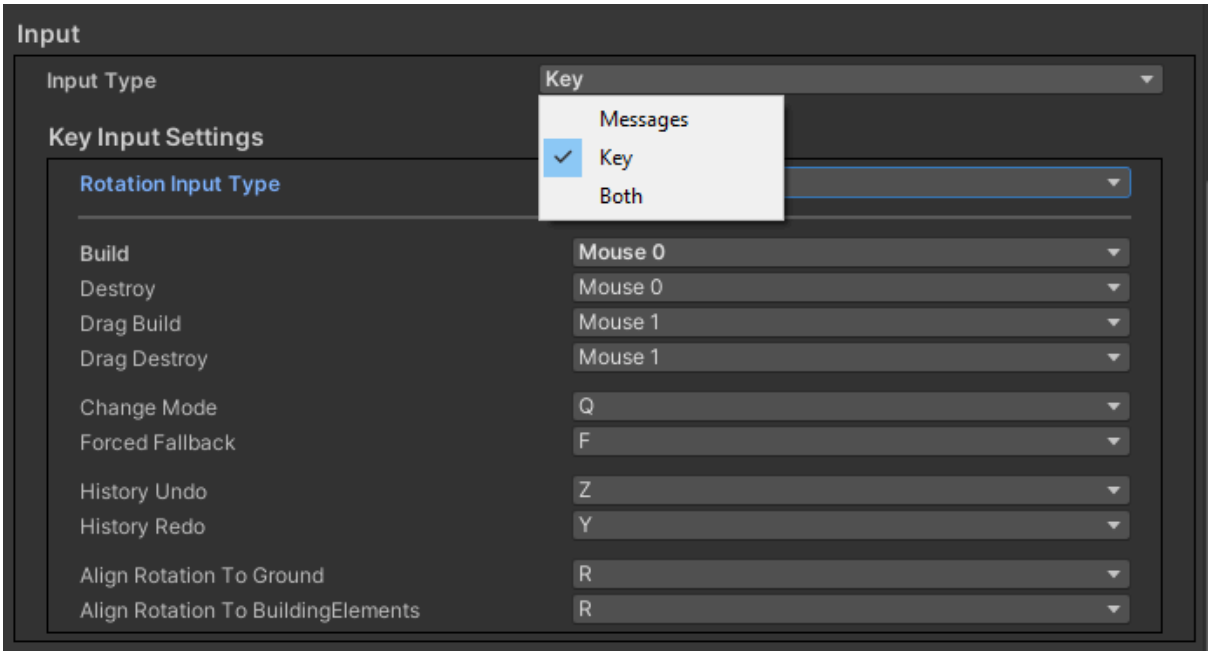
process. For example if the element has set the AdvancedGird building algorithm and the Forced Fallback is enabled then the fallback FreeBuilding algorithm will be used.
    - Note that this feature is connected to a button press to activate the feature. So the feature should be enabled by the developer on the BuildingManager's UI with this boolean. Then during runtime the default algorithm will be used until the player pressed the set button to activate or deactivate this feature. The activation state is set to false in case of
        - The BuildingManager has been reactivated.
        - The BuildingManager has been deactivated.
        - The BuildingMode is changed (built to destroy or destroy to build).
        - And a special case when an element is placed. This is a special one that can be turned on or off with another boolean: Forced Fallback Reset On Element Place.
    - Note that because the fallback algorithm is the FreeBuilding by default this forcedFallback has to have no effect on those elements that are using the FreeBuilding as main algorithm.
- Forced Fallback Reset On Element Place.
    - This boolean is only available when the Enable Forced Fallback is true so when the Forced Fallback feature is available.
    - If it is true then the ForcedFallback feature's active state is reset to false when an element is placed.

Raycast



- Raycast Mode
    - A Camera or the cursor should be used for raycast.
- Raycast Settings
    - The object holding the raycast settings for Manager.

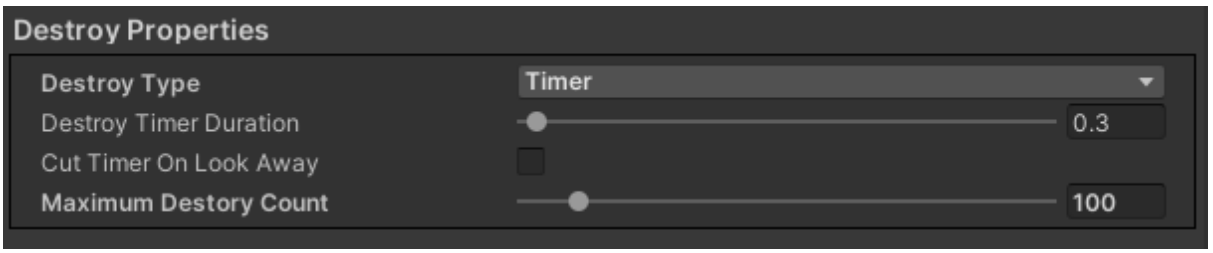
Input



- Input Type
    - You can set the input type for the BuildingManager.
    - New, Legacy, Both
    - In case of Legacy or Both you should provide the Legacy Input Settings which are keybindings for the inputs

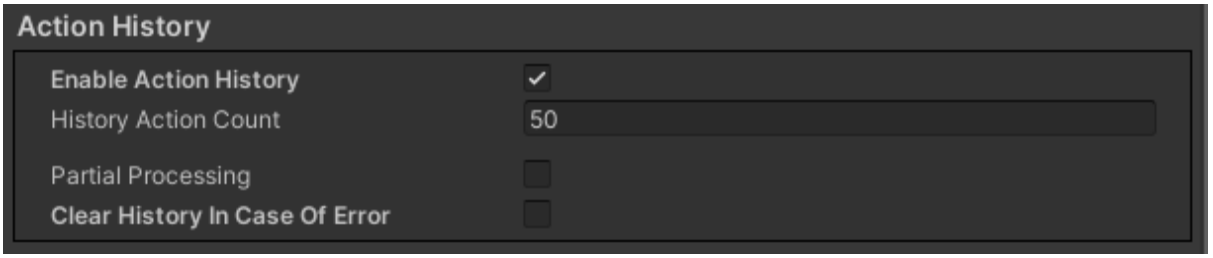
The Input settings will be explained later in this document.

Destroy Properties



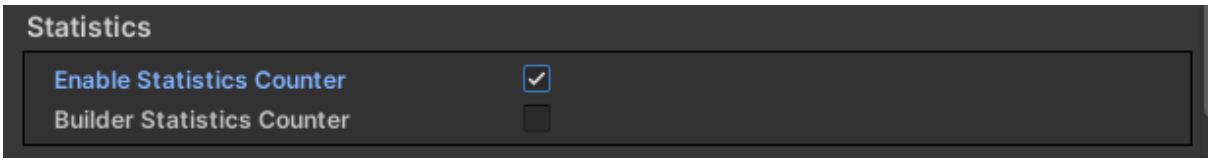
- Destroy Behaviour
    - How should they destroy work? It is explained later.

Action History



- Enable Action History
    - Enable or disable the Action History feature.
- History Action Count
    - How much action is saved by the History feature
- Partial Processing
    - In case of error allow for the History feature to try to undo or redo the actions partially if it is possible.
- Clear History In Case Of Error
    - If any error happened, clear the whole history.

Statistics



- Enable Statistics Counter
    - Enable the statistics measurements for the Manager only.
- Builder Statistics Counter
    - Enable the statistics measurements for the Builder algorithms too.

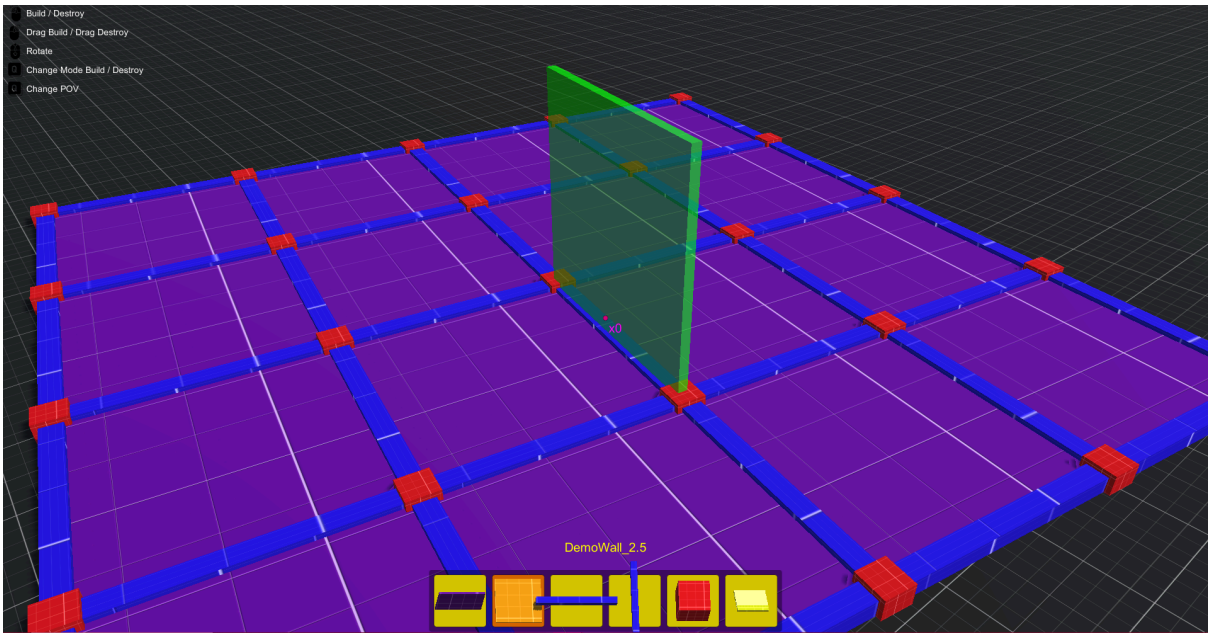
Main Features

SimpleBuild, *DragBuild*, *SimpleDestroy*, *DragDestroy*

These four are the main features of the BuildingManager. These features are required for giving the developers a possibility to place and remove BuildingElements in the scene.

Simple Building

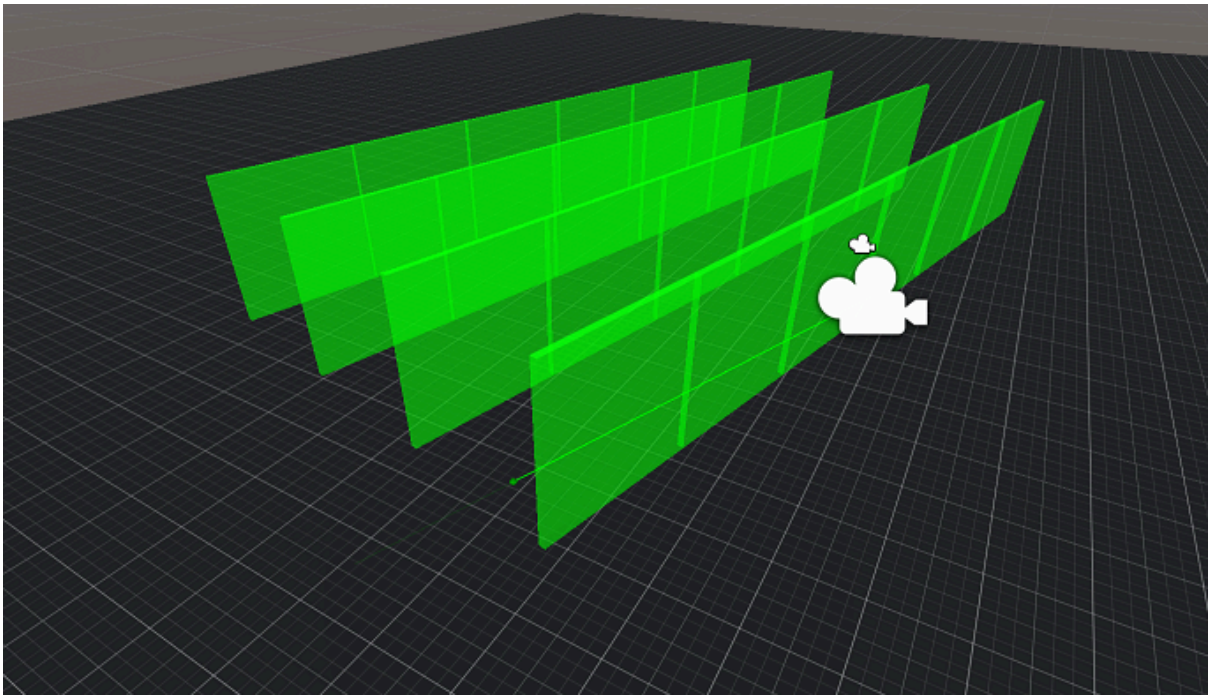
The main goal of the BuildingManger is implementing an in-game modular building tool for developers. What does it mean? The BuildingManager provides an algorithm that the developers can use to make games where the player can build such things in games like buildings, houses and other Structures.
The SimpleBuilding feature is responsible for that the players are able to build a structure or anything as part by part like walls, floors, roofs etc. The SimpleBuilding can only build one element at a time.
During the update after the raycast the BuildingManager tries to find the best position to build the BuildingElement. if it is possible the position search algorithm gives a position based on the specifications where a temporary element will be placed. If it is good to the player the element can be placed permanently there.

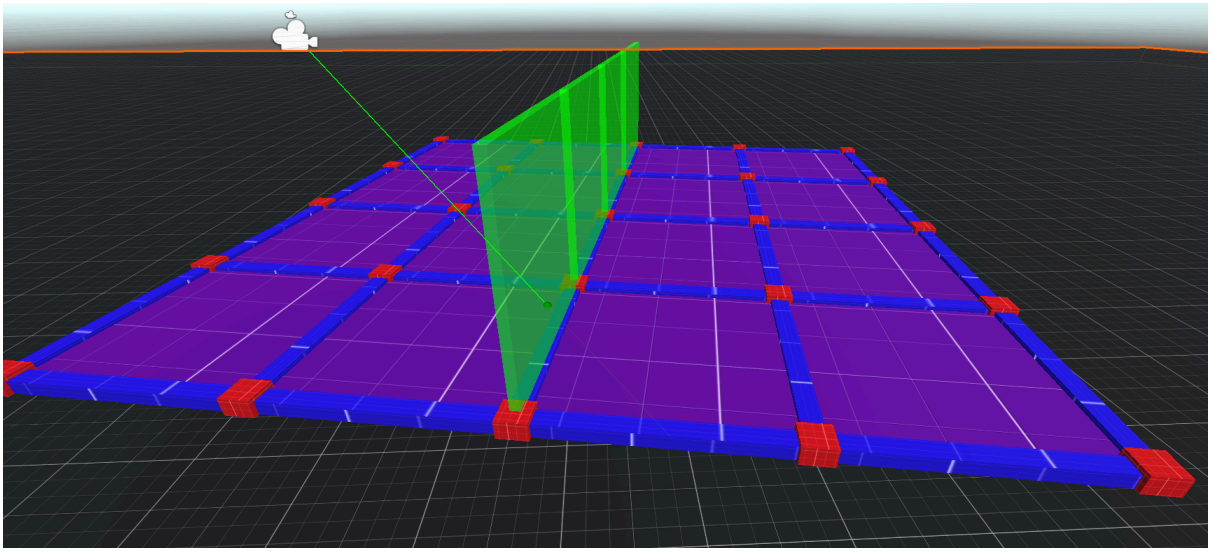


Drag Building

Building a massive structure piece by piece can be very slow. For making the players' life easier the BuildingManager provides a DragBuilding functionality that is responsible for placing multiple elements at once.
During the Update when the position is found by the SimpleBuilding. The player can build the element in a simple way with the left mouse button but if the player holds the right mouse button the BuildingManager changes to DragBuilding mode.
During the DragBuilding mode the manager will use the raycast to know where the TemporaryBuildingElements should be placed. The manager will copy the elements next to each other to where the raycast points to. If the player releases the right mouse button every BuildingElements that have a valid position will be placed. The validity of the positions based on the settings.

Every Building algorithm behaves different ways during the drag building.

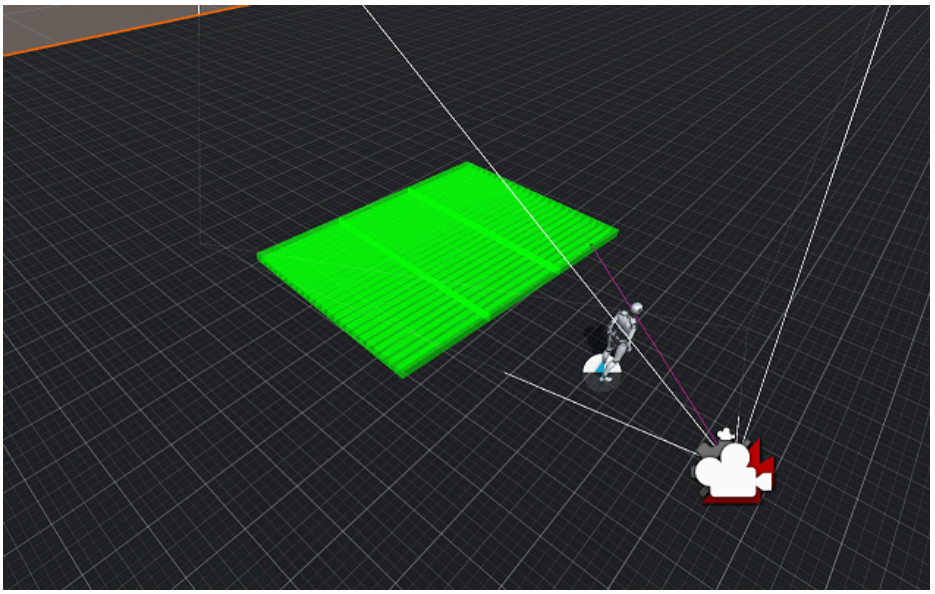




The algorithms will be explained in detail in the **Building Algorithms** chapter. Now let's just see the difference between the algorithms during the drag building.

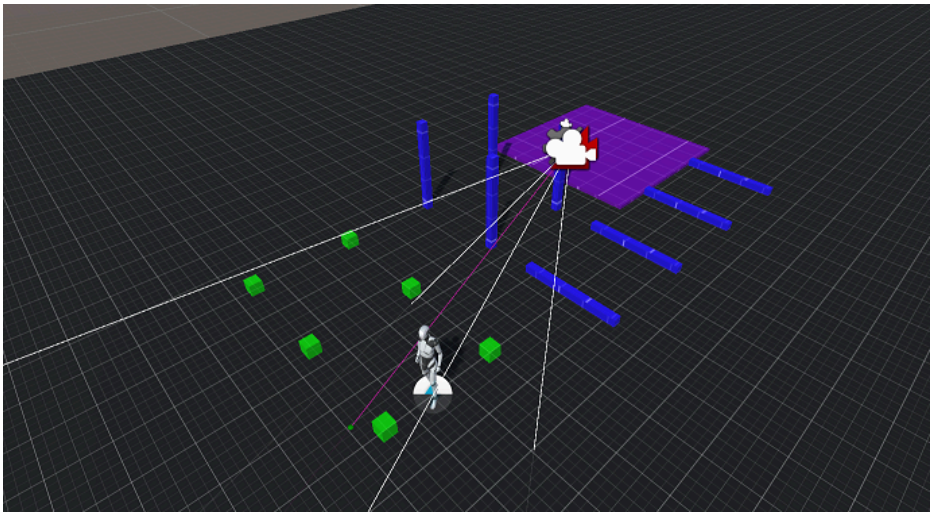
FreeBuilding

The FreeBuilding using the BuildingElment's Dimension for calculating the TemporaryBuildingElements' position.
In this example the BuildingElement's dimension is (0.2 x 0.2 x 2.5) what is placed with DragBuilding + FreeBuilding.

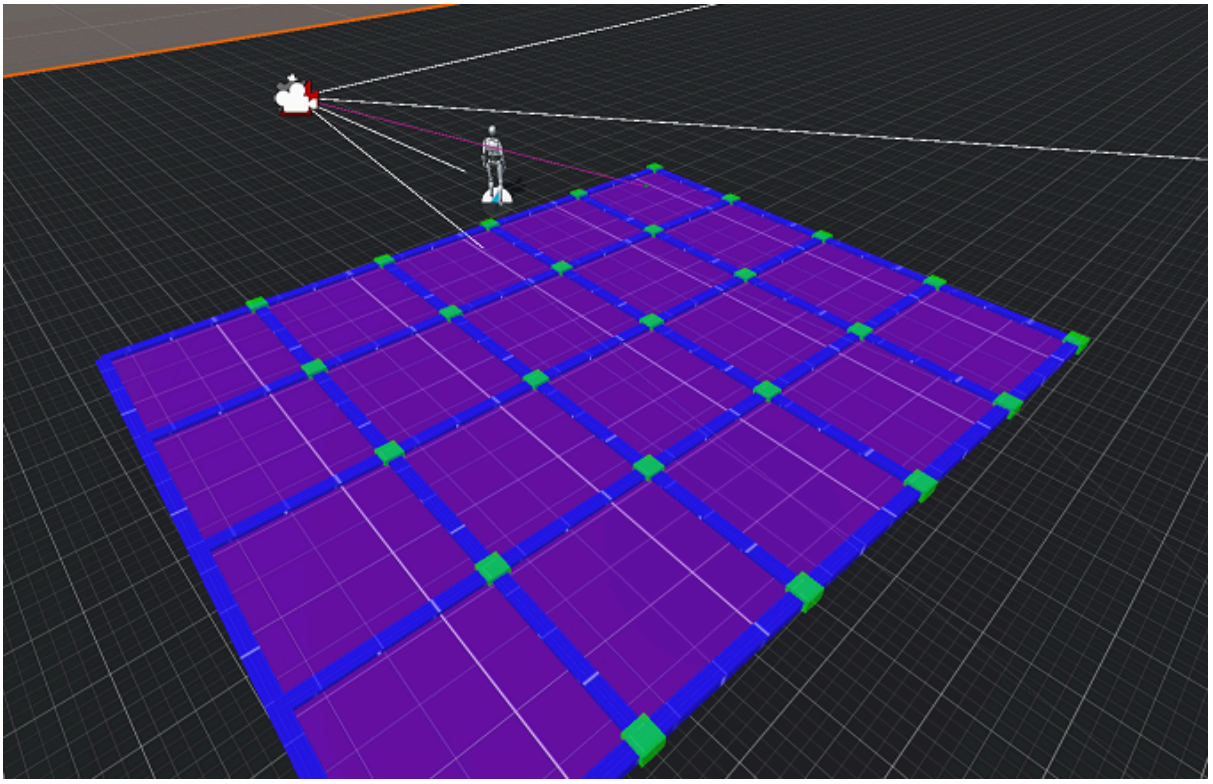


BasicGridBuilding

The GridBuilding is using the size of the Grid to calculate the TemporaryBuildingElements' position. The elements are placed in the middle of the grid's square.



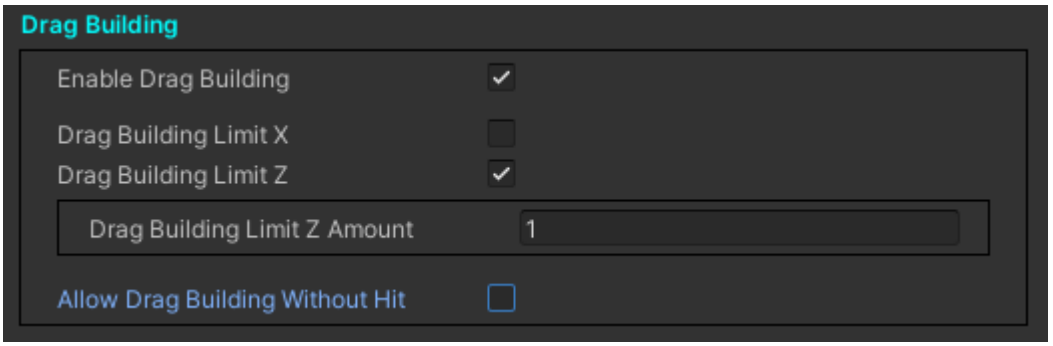
AdvancedGridBuilding
The AdvancedGridBuilding is using the grid's size but it is snapping the elements on different positions based on their types.



(Here the green cubes are placed with dragbuilding. These cubes are signed as corners so these should be snapped on the corner of the grids.)

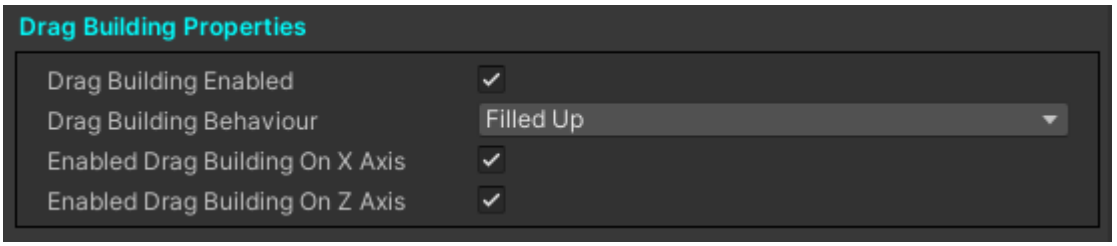
SnapPointBasedBuilding
Currently this feature is not supported by the SnapPointbasedBuilding algorithm.

Settings by the Algorithms



The Algorithm's Drag Building Setting is explained at the Settings' Drag Building section in another documentation.

Settings by the BuildingElements



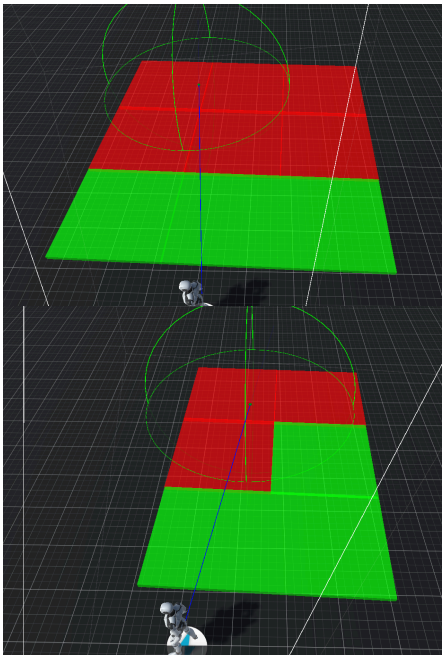
Also the DragBuilding's behavior can be changed by the BuildingElement too.
These Settings are available on the BuildingElement's MetaData. Every MetaData can decide that it's BuildingElements can be used for drag building or not or it can limit the axes too or change it's behaviour.
Note: even if you enable the drag building but the element will disable its possibility for drag building the BuildingElements can not be for drag building. Or if you limit one of the axes on the BuildingManager but the other one is limited by the BuildingElement the whole drag building will not work because all axes are limited.

Material Functionality

Base Concept

Every building system uses a material functionality like if you want to build a stone wall you need to have an amount of stone to do that. This BuildingManager can achieve that with its maximum amount of placeable item count functionality. You can set a number which means how many BuildingElements can be placed at the same time with the Drag Building feature.
For example: you set the maximum number to 6 then if the player wants to place more than six elements with the Drag Building feature every element above 6 will be Blocked.
Note that this number can be zero so every element during the drag building will be blocked even if you are using just the simple building that one element will be blocked too. This mimics that when the player has not enough material neither for one element.

Refresh Blocking



The Blocked/Active element's count should be refreshed if you refresh the number with the **IBuildingManagerExternalInterface**'s **SetMaximumBuildingElementCount** function. If this function is called and the given number is different from the previous the block state of the elements will be refreshed.
Also the blocking state can be refreshed every time when the area has changed so new rows/columns are added or removed.
The Algorithm tries to fill up every row with Active elements and Block the others. It can cause different shapes based on the dimension and the amount of the temporary elements dragged area. Check the image above.

Manage MaximumBuildingElementCount

You can manage this number through the **IBuildingManagerTracker** and the **IBuildingManagerExternalInterface** interfaces with the following functions:

- IBuildingManagerExternalInterface
    - SetMaximumBuildingElementCount
- IBuildingManagerTracker
    - GetMaximumCountOfTMPElements

With the **SetMaximumBuildingElementCount** this maximum number can be changed from outside the BuildingManager.
The BuildingManager is using the **GetMaximumCountOfTMPElements** function to get this number if it needs to refresh that number. If the tracker is null or Sandbox boolean (On the BuildingManager script's interface) is set to false the number will be always uint.MaxValue.

Sandbox mode

You can set a boolean called Sandbox. If it is true then the whole material functionality is turned off and the algorithm is ignoring the set number. In the case of Sandbox, true every TemporaryBuildingElement should be Active. (It can be Blocked because of other reasons but not because of the MaximumNumber)

Action History

The main goal of the Action History feature is to give the possibility to the players to undo or redo their actions. The Advanced Building System's has basically two kinds of actions, build and destroy.

This feature is an optional feature so the developper can decide to turn on or off this functionality. This feature is disabled by default. The history has some negative effects like increased memory usage and increased the created objects count. It can destroy or create a huge amount of elements so in case of non optional settings the undo or the redo actions can highly increase the calculations of a frame.

The Action History feature should support and contribute with all of the other features of the Advanced Building system. For example it can undo or redo a build action of a prebuilt element or an overridden element. It can handle the final element and it can handle the undo or the redo actions for the DragBuilding or the DragDestroy.

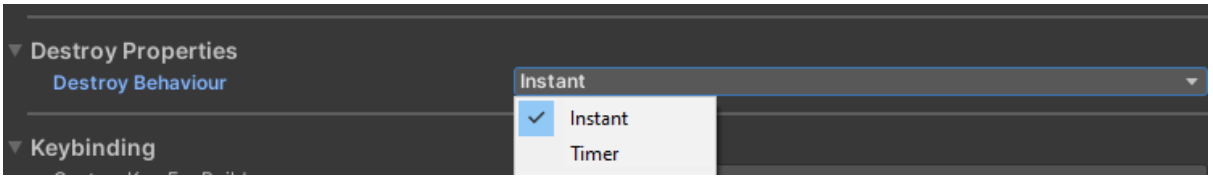
The History feature stores and handles the action in a stack like way so the last action can be undo. It stores the undid actions and the player can redo them backwards. If the player undo some actions and do a new one the History feature will destroy all of the undid action and they can't be redone any more and the new action added to the stack. The feature has a customisable maximum count so the amount of the actions stored by the History reaches this maximum number in case of a new action the first added action will be removed.

The feature doesn't require special settings, just a few parameters.

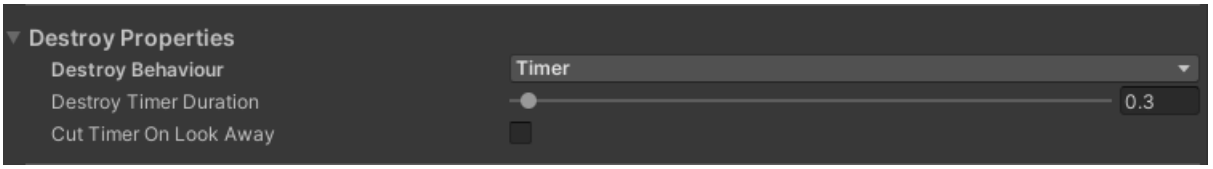
The action history provides some possibility through the provided trackers to ensure that the developers have the maximum control on the game. The feature can handle the action of the BuildingManager but there can be some scenarios when the undo or the redo steps are not possible because of unrelated factors that can not be managed by the asset's algorithms. Like an element destroyed by outside of the Advanced Building Systems logic. In case of an error the action is impossible to be Undo or Redo so it will be deleted.

Simple Destroy

The BuildingManager gives a removing feature too. It works in such ways when you change to Destroy mode with the “**Q”** button then the raycast will be used to track which element would be deleted by the player. The player looking to a BuildingElement will be highlighted and then with the mouse’s left button the element can be deleted.
The destruction can happen in an instant way or it can have a little delay in what can be set up on the UI. Here the time should be chosen and then we can set a float as seconds how much time should be held by the player to destroy the element.

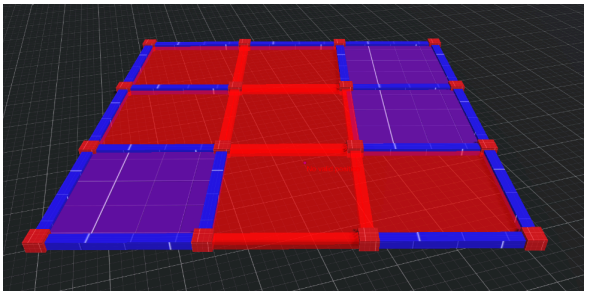


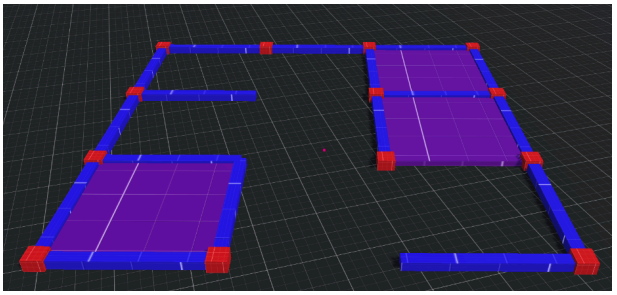
If the time is used then is it questionable the player should be looking at the element for the whole time during the set time? It can be set on the UI too. If the “Cut Timer On Look Away” property is set then if the player looks away from the BuildingElement the destruction is aborted. If it is unset then even if the player is looking anywhere else the destroy will be finished if the button is held for the required time.



Drag Destroy

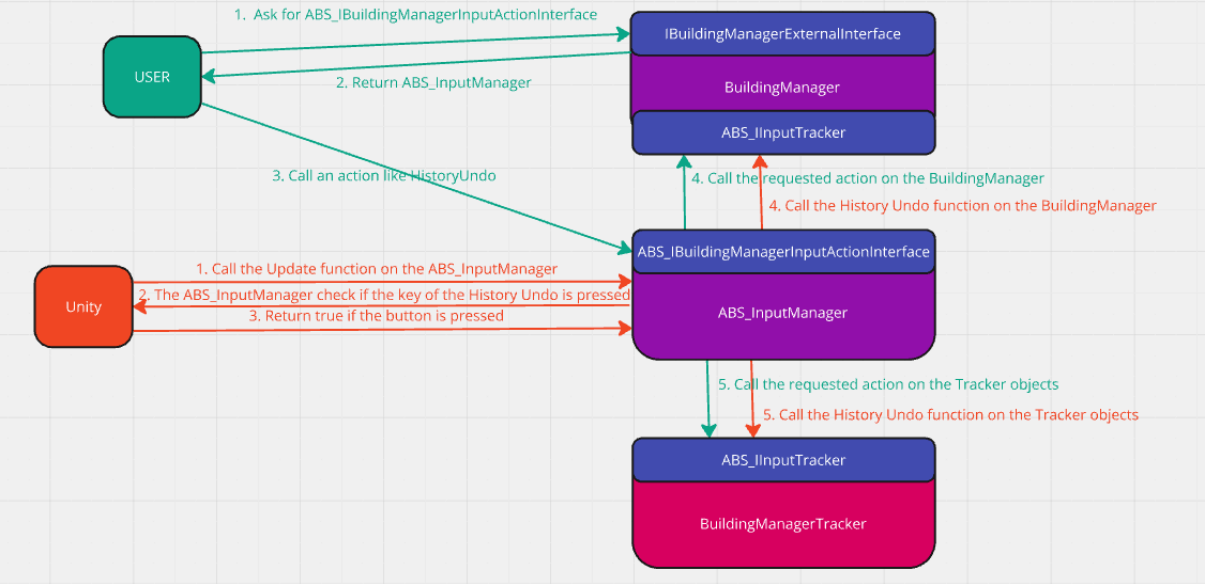
The BuildingManager makes it possible to destroy more elements at one time. The player should hold the right button of the mouse to achieve this functionality. Every element will be signed for destruction during the time when the player holds the button. Every element will be signed what had been hit by the raycast. If the player wants to finalize the destroy the left button should be pressed too (or hold if the timer is set up).





Input

The BuildingManager will handle the inputs of the player with the ABS_InputManager component responsible for handling the legacy key based inputs and the messages or both at the same time.



The InputManager can be obtained from the BuildingManager through the IBuildingManagerExternalInterface. The BuildingManager will give back the InputManager as ABS_IBuildingManagerInputActionInterface which is responsible for handling the input of the Advanced Building System. With that interface you can change the settings of the input system during runtime like the keybindings of the inputs. Also this interface can be used for sending message inputs for the InputManager which will forward the inputs to the BuildingManager.

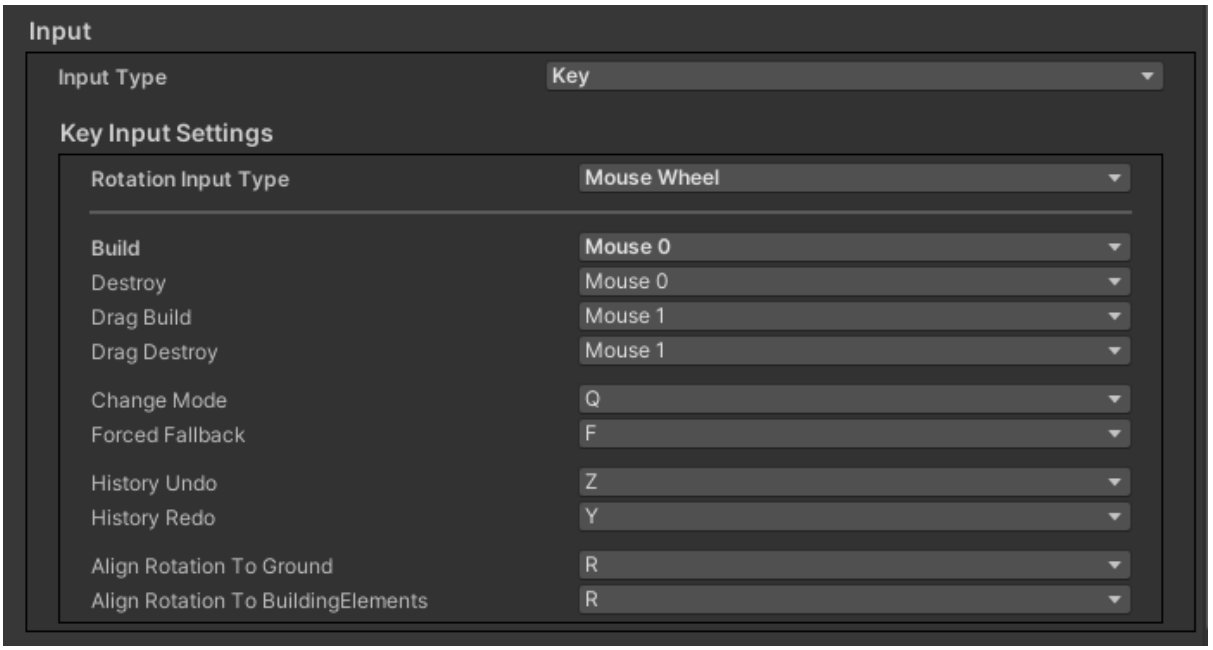
```
public interface IBuildingManagerExternalInterface
{
    //+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
    //  Input
    //+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

    /// <summary>
    /// Getter for the Input Interface of the BuidlingManager
    /// </summary>
    /// <returns> The Input Interface of The BuildingManager.</returns>

    public ABS_IBuildingManagerInputActionInterface GetInputInterface();
```

The Input system will send all of the inputs to the tracker objects too. The BuildingManagerTracker is implementing the ABS_IInputTracker interface and it will get all of the input right after the BuildingManager so the developers can react to the same inputs as the Advanced Building System. The BuildingManager gets those inputs through the ABS_IInputTracker interface too.

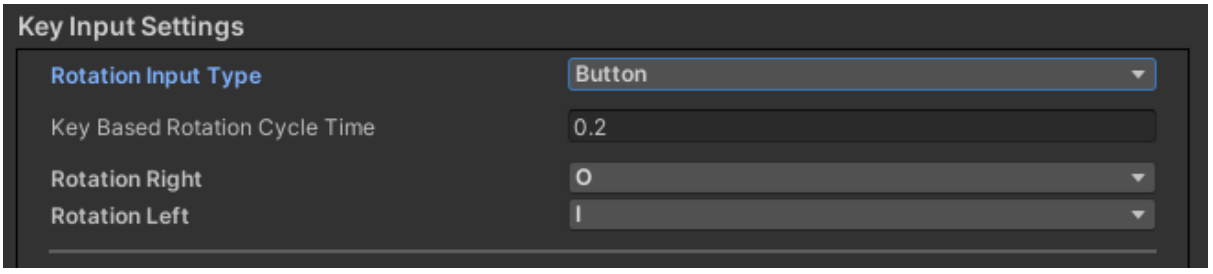
The key based legacy inputs are still available and it can be set on the GUI of the BuildingManager.
First of all you can set the InputType :

- Key
- Message
- Both

The following settings can be only set if the Key or the Both input type has been chosen.



- Rotation Input Type
    - The rotation by default is handled by the mouse wheel. but it can be changed to Button setup too.

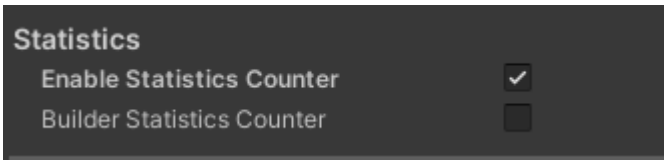


    - In case of the Button setup 2 input key should be set the
        - Rotation Right
        - Rotation Left
    - If those buttons are pressed 1 rotation unit is applied on the Building Element.
    - If the buttons are held then the rotation is repeated in a cycle that will happen in every 0.2 second. This value can be changed with the “Key Based Rotation Cycle Time” float value next to the input key settings.
    - Note that if both rotation input buttons are pressed and held by the player then no rotation has been applied on the element.
- Build
    - For the Simple Build the default key is the **Mouse 0**.
- Destroy
    - Originally the same button is used for these two features because these 2 are

the basic functionality of the BuildingManager just in different modes.
    - **Mouse 0** by default
- Drag Building
    - The default button is **Mouse 1**
- Drag Destroy
    - Similarly to the Simple Destroy this feature's default button is equal with the Drag Building.
    - But the possibility is available to change it and make it different from the other features.
    - **Mouse 1** by default
- Change Mode
    - Changing mode between destroy and build can be done by default with **Q**.
- Forced Fallback
    - Activate or deactivate the Forced Fallback feature.
- History Undo
    - If the Action History feature is enabled with this input the last action can be undo.
- History Redo
    - If the Action History feature is enabled with this input the last undone action can be redo.
- Align Rotation To Ground
    - With this button you can enable or disable the rotation alignment to the ground feature. More details at the rotation settings.
- Align Rotation To BuildingElement
    - With FreeBuilding the rotation can be aligned to match another BuildingElement with this set Button.

Statistics

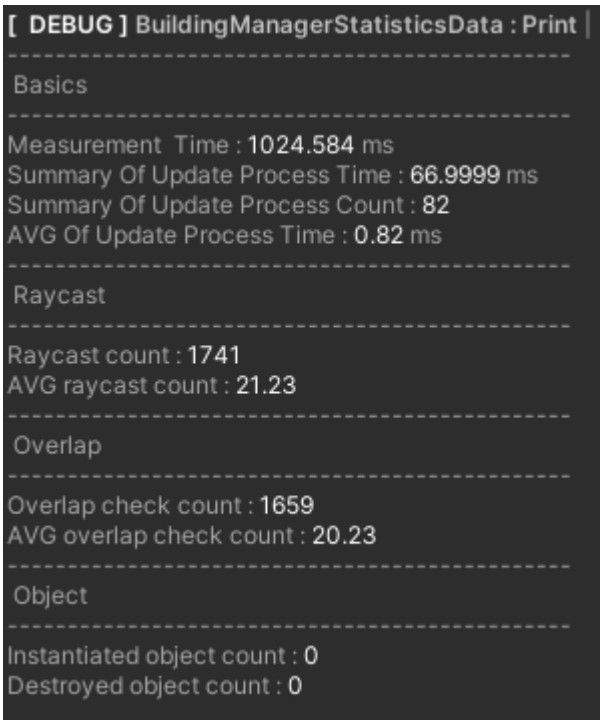
Note that every Statistics related part of the code is behind the UNITY_EDITOR macro which means that it should not be compiled into the game. So these functionalities should have zero impact on the final compiled game!



On the BuildingManager's UI you can enable the "Statistics Counter". The manager will measure its performance for one second and write out to the log some statistics.

Currently the following messages can be written out by the manager:

BuildingManager's statistics.



This log contains basic measurements about the manager's Update function.

1. Measurement Time
   The exact 1 sec of the measurement based on the Delta Time.
2. Summary Of Update Process Time
   How much time is exactly measured.
3. Summary Of Update Process Count
   How many updates were processed during the measured time.
4. AVG Of Update Process Time
   Average process time of the update function.
5. Raycast count
   How any raycast performed during the measured time.
6. AVG raycast count
   Average amount of the raycasts during a single frame.
7. Overlap check count
   How many overlap checks performed during the measured time.
8. AVG overlap check count
   Average amount of the overlap checks during a single frame.
9. Instantiated object count
   How many objects created by the BuildingManager during the measured time.

10. Destroyed object count
    How many objects destroyed by the BuildingManager during the measured time.

AdvanceGrid Search Statistics

This message contains some statistics about the calculations of the AdvanceGrid Search's algorithm. This is only written out if the AdvancedGrid Search has been used.

1. Summary Of Position Process Count
   How much position search is requested by the BuildingManager. One search call should happen in one frame.
2. Summary Of Position Process Time
   The summary of the time spent by the algorithm to calculate the position.

3. AVG Of Position Process Time
   On average how much time was required to calculate a position.
   Note that this average contains the high calculating processes and those that used cache.

4. Maximum Position Process Time
   The longest position process time during the measurement.

5. Summary of checked Buildings
   The summary of the checked Buildings.

6. AVG of checked Buildings
   On average how much Building had been processed per search.

7. Summary of checked BuildingElements
   The summary of the checked BuildingElements.

8. AVG of checked BuildingElements
   On average how much BuildingElements had been processed per search.

9. Summary of High Impact SnapPoints
   How much SnapPoints had a high performance impact. A SnapPoint is treated as high impact if it had been validated. The validation process required some high calculation based logic that can have an impact on the performance such as raycast, collision check and a lot of iteration on different containers. For better performance the BuildingManager tries to minimize the amount of these SnapPoints. It can be achieved by finding the good (not the best) valid SnapPoint as fast as possible because after that it can discard most of the SnapPoints by the range.

10. AVG of High Impact SnapPoints
    On average how much SnapPoints was High Impact.

11. Summary of Successful Validation SnapPopint
    How much SnapPoint validated successfully. The “High Impact SnapPoint” shows how much SnapPoints were validated and this counter shows how much was successful from these.

12. AVG of Successful Validation SnapPopint
    On average how much SnapPoint had been successfully validated.

13. Summary of Failed Validation SnapPopint
    How much SnapPoint validation failed.

14. AVG of Failed Validation SnapPopint
    On average how much SnapPoint validation failed.

15. Summary of Failed Custom Validation SnapPopint
    How much SnapPoint validation failed on custom validation. The custom validation is a function call on the Tracker object to give the possibility to deny the building by the developers own logic. The custom validations counted into the failed validation too.

16. AVG Failed Custom validation
    On average how many position custom validation fails.

17. Successful first SnapPoint check : X/Y
    On average how many First SnapPoint checks were successful. Before the manager tries to calculate the SnapPoints it tries to validate the nearest SnapPoint. If it was

valid then the process of the Building stops because this SnapPoint should be the nearest one. Every Other one can be discarded. The X is the count of all processed Buildings and the Y is the time when the first SnapPoint check was successful.

18. Successful Positioning Using Cache : X/Y
    On average how much Building processing was done by the cached data. The X is the count of all processed Buildings and the Y is the time when the cached data was used.

Tracking

The BuildingManager is a standalone module that doesn't need to be managed by other objects. You as a developer just need to set up the BuildingElement and settings. The Manager will work on by itself and doesn't need to be managed but sometimes it needs some information or some data from the project. To ensure the manager works to its maximum potential an object that implements the **BuildingManagerTracker** which means that you have to inherit from the BuildingManagerTracker class and override its functions. The Manager will call it's function for:

- get information
- give you information about the state of the building process
- give you the possibility to make decisions to change the behavior of the Manager (allow or deny actions)
- give you the possibility to handle or react to events happened during the build process

The BuildingManagerTracker implements the ABS_IInputTracker interface. Which means that through the tracker the BuildingManager provides the possibility to react for the same inputs as what the manager got.

Note that the BuildingManager can handle multiple trackers at the same time.

BuildingManagerTracker functions

- ExternalInterface
    - SetExternalInterface
        - This function is called from the Awake of the BuildingManager. It will give you the Manager as **IBuildingManagerExternalInterface** interface. This interface will be explained later.
- Building Element
    - ResetBuildingElement
        - This function is called when the BuildingElement is changed to a different one.
        - It is called too when the BuildingElement is changed to null.
    - BeforePlace
        - This function is called when the player triggers the placing of a BuildingElement. It is called for every single BuildingElement during the DragBuilding too.
        - This function should return a boolean value. If the value is false the BuildingElement will not be placed.
        - It will give you the possibility to create custom logic and decide that a BuildingElement can be created or not so you can override/expand the logic of the BuildingManager
    - BuildingElementPlaced
        - This function is called after a BuildingElement is placed. It will be called once during the DragBuilding too.
- Building
    - BuildingPlaced

        - This function is called when a new Building is created by the Manager
    - BuildingWillBeDestroyed
        - This function is called when a Building will be destroyed.
- Building Cycle
    - StartOfBuildingCycle
        - It is called at the start of the building process in every update function. Basically it is called at the start of the Update function.
    - EndOfBuildingCycle
        - It is called at the end of the building process in every update function. Basically it is called at the end of the Update function.
    - RaycastOnHit
        - This function is called when the raycast of the BuildingManager hits something.
- AdvancedGrid and BasicGrid Building
    - PositionCustomValidation
        - It will be called during the AdvancedGridBuilding process. When it validates a snap point position the function has been called.
        - A boolean value should be returned. If the boolean is false the position is dropped by the algorithm.
        - It gives you the possibility to implement a custom snap point validation logic.
- Temporary Building Process
    - DragBuildingStarted
        - It is called when the drag building is started.
    - DragBuildingStoped
        - It is called when the drag building is stopped.
    - FirstBuildingElementBlockedStateChanged
        - It is called when the element used for SingleBuilding (the first element of the drag building) changes its state.
    - FirstBuildingElementCurrentPosition
        - It is called in every frame to refresh the current position of the first temporary element.
    - CurrentValidBuildingElements
        - It is called during the drag building when the number of the temporary elements changes (new line is added or removed).
    - GetMaximumCountOfBuildingElements
        - It is called to get information about how many elements can be placed at once during the drag building.
        - It should return a number.
        - It is explained in detail in the Material Functionality chapter.
- Destroy Process
    - BeforeDestroy
        - It is called when the player triggers the destruction of an element.
        - It should Return a boolean. If the boolean is false the element will not be destroyed.
        - It is called for every single element during the DragDestroy feature too.
        - It gives you the possibility to override the logic and create custom logic

to avoid the destruction of elements.
    - DestroyTimerStarted
        - This function is called when the destroy timer is started.
        - It is not called if the destroy is set as Instant
    - DestroyTimerIsOngoing
        - It is called during the destroy process when the timer is started.
        - It is called in every frame to refresh the leftover time.
    - DestroyTimerStoped
        - It is called when the timer has stopped.
    - BuildingElementDestroyed
        - It is called right before the BuildingElement is destroyed. Here you have no possibility to cancel the destruction of the element anymore.
- Action History
    - BeforeHistoryDestroy
        - This function is called when an element will be destroyed by the Action History feature.
    - BuildingElementHistoryDestroyed
        - This function is called when an element has been destroyed by the Action History feature.
    - BeforeHistoryPlace
        - This function is called when an element will be placed by the Action History feature.
    - BuildingElementHistoryPlaced
        - This function is called when an element has been placed by the Action History feature.
    - BuildingWillBeHistoryDestroyed
        - This function is called when a building will be destroyed by the Action History feature.
    - BuildingHistoryPlaced
        - This function is called when a building has been placed by the Action History feature.

The ABS_IInputTracker interface's functions are not explained in detail. When an input happens the InputManager will call these functions right after the BuildingManager. The BuildingManager gets those inputs through the ABS_IInputTracker interface too.

IBuildingManagerExternalInterface functions

- Input
  - GetInputInterface
    - Return the ABS_InputManager as ABS_IBuildingManagerInputActionInterface.
- Tracker
  - AddTracker
    - With this function a new tracker can be added to the tracker list.
  - RemoveTracker
    - With this function a tracker can be removed from the tracker list.
  - RegistrateStatTracker (Editor Only[1])
    - With this function a new Statistics tracker can be added to the Statistics tracker list.
  - UnRegistrateStatTracker (Editor Only)
    - With this function a Statistics tracker can be removed from the Statistics tracker list.

```
#if UNITY_EDITOR
        /// <summary>
        /// Add a new statistics tracker to the BuildingManager.
        /// </summary>
        /// <param name="p_Tracker">The new tracker added to the statistics tracker's lists.</param>
        2 references | Changed by resb014@gmail.com on Friday, September 27, 2024
        public void RegistrateStatTracker(in IBuildingManagerStatisticsTracker p_StatTracker);

        /// <summary>
        /// Remove one of the statistics trackers of the BuildingManager.
        /// </summary>
        /// <param name="p_Tracker">The tracker should be removed from the statistics tracker's lists.</param>
        5 references | Changed by resb014@gmail.com on Friday, September 27, 2024
        public void UnRegistrateStatTracker(in IBuildingManagerStatisticsTracker p_StatTracker);
#endif
```

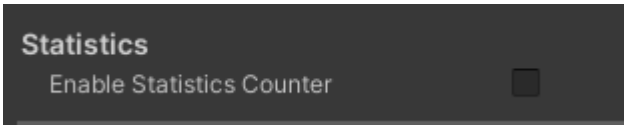
- BuildingManager handling
  - Activate
    - Call it to activate the BuildingManager
    - It has 2 overload
      - One required a number which is the index of the BuildingElement from the given BuildingElementList
      - The other one needs a BuildingElement what will be used for building
    - Both versions have a BuildingManagerBuildMode property which will determine if the building process will be continuous or single build.
  - IsActive
    - Return true if the BuildingManager is active.
  - Deactivate
    - Call is to stop the BuildingManager
  - ChangeMode

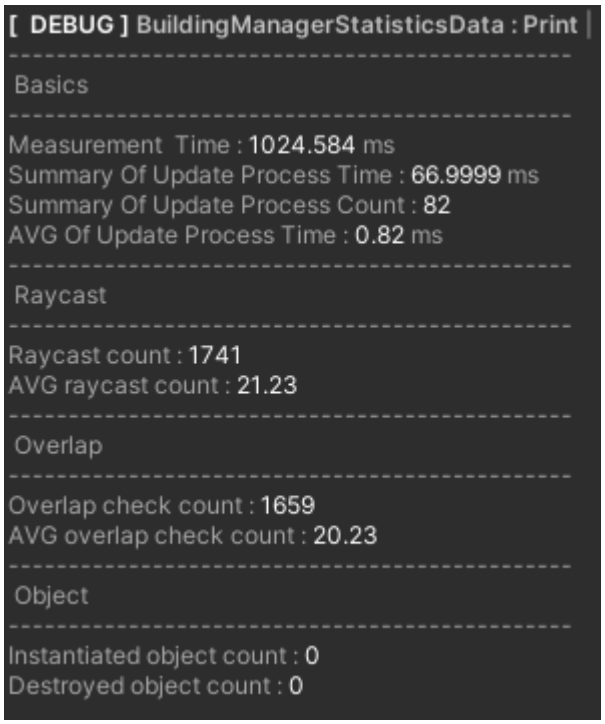
[1] Editor Only feature means that it is behind the UNITY_EDITOR interface. The statistics functionalities are not compiled into the final product.

        - The BuildingManager has 2 modes. Build and Destroy. Usually the player can trigger the change of the modes by pressing the set button but this function gives you the possibility to change it from your algorithm based on your logic.
    - GetMode
        - Return the current mode of the BuildingManager.
    - SetBuildingElementList
        - Reset the list of the BuildingElements.
        - with 2 overloads
            - One just required the list and only change the list of the manager
            - The other one required an index and the rest of the BuildingElement too, not just the list.
    - GetActiveBuildingElement
        - Return the active BuildingElement or return null if the manager isn't active.
- Raycast
    - SetRaycastCameraSettings
        - With this function the raycast setting can be changed.
        - It can be useful when the game changes its point of view.
    - ResetCamera
        - Change the camera. Also useful during the POV change as SetRaycastCameraSettings.
        - Note that the setting does not contain the camera so if a setting with cursor setup changes to a settings containing a camera setup the camera should give through this function.
- MaximumBuildingElementCount Handling
    - EnableSandbox
        - Change the BuildingManager into sandbox mode.
    - DisableSandbox
        - Disable the sandbox mode.
    - SetMaximumBuildingElementCount
        - Set a new maximum amount to the Manager. Check the Material Functionality chapter for more details.
- Performance
    - ClearCache
        - Clear all the Cache of the BuildingManager's.
        - Note that this is only covering the BuildingManager's caches the Buildings' have their own caches and they are not cleared by this function call!
    - ClearHistory
        - Clear the actions from the Action History feature.

IBuildingManagerStatisticsTracker functions

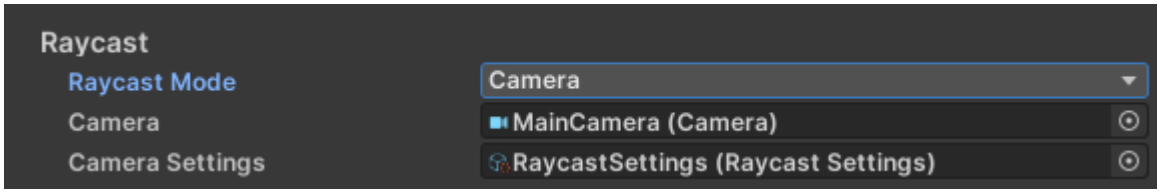
- StatisticsDataCallback
    - If the “Enabled Statistics Counter” is true on the interface of the BuildingManager the manager will print out statistics about the manager’s algorithm. The details of the data will be explained in detail in the Statistics section of his documentation. If the statistics counter is enabled the Manager will call your StatisticsTracker’s StatisticsDataCallback function with a BuildingManagerStatisticsData object containing the data that is written out by the manager. The Statistics tool of the manager is based on this interface too. Note That this is an Editor Only feature which means that it is behind the UNITY_EDITOR interface. The statistics functionalities are not compiled into the final product.



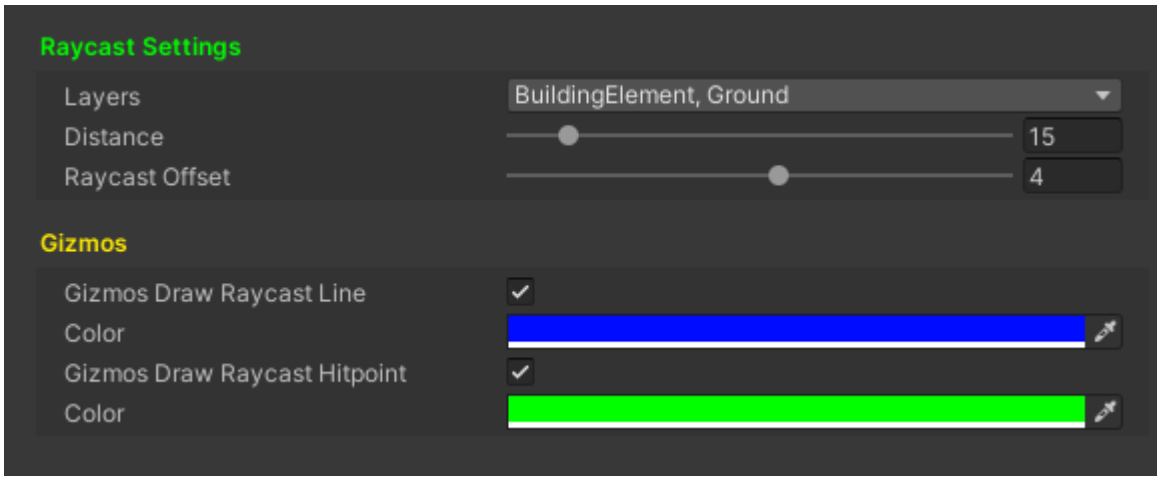


Raycast

The whole AdvancedBuildingSystem is based on the raycast. The Manager performs a raycast every frame and tries to find the best position based on the setting. But for that the raycast should be set up too.

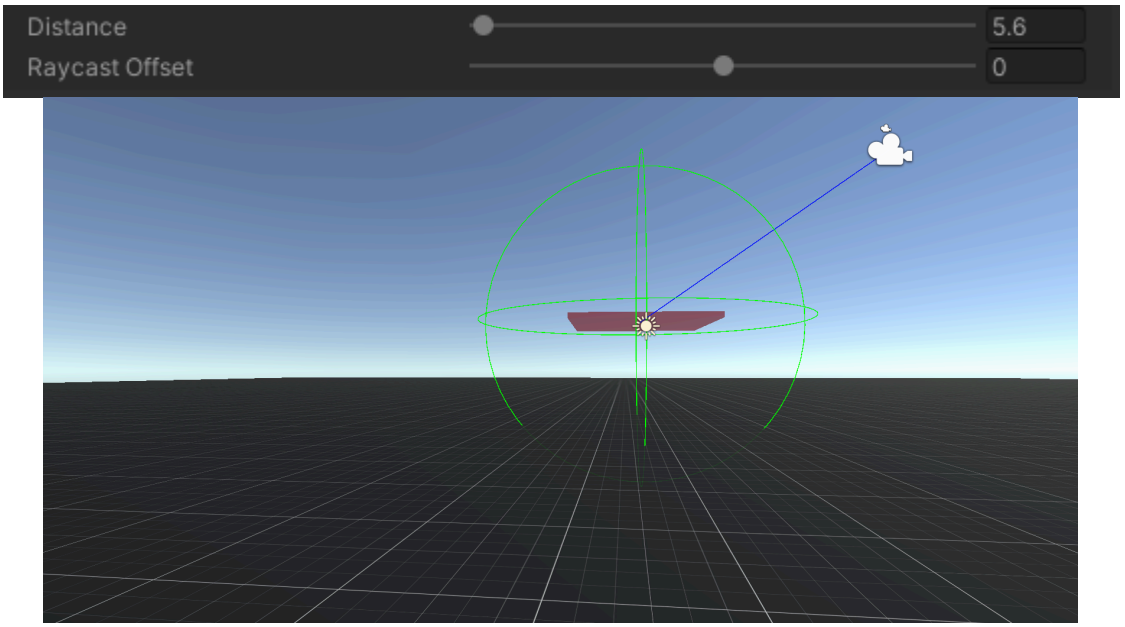


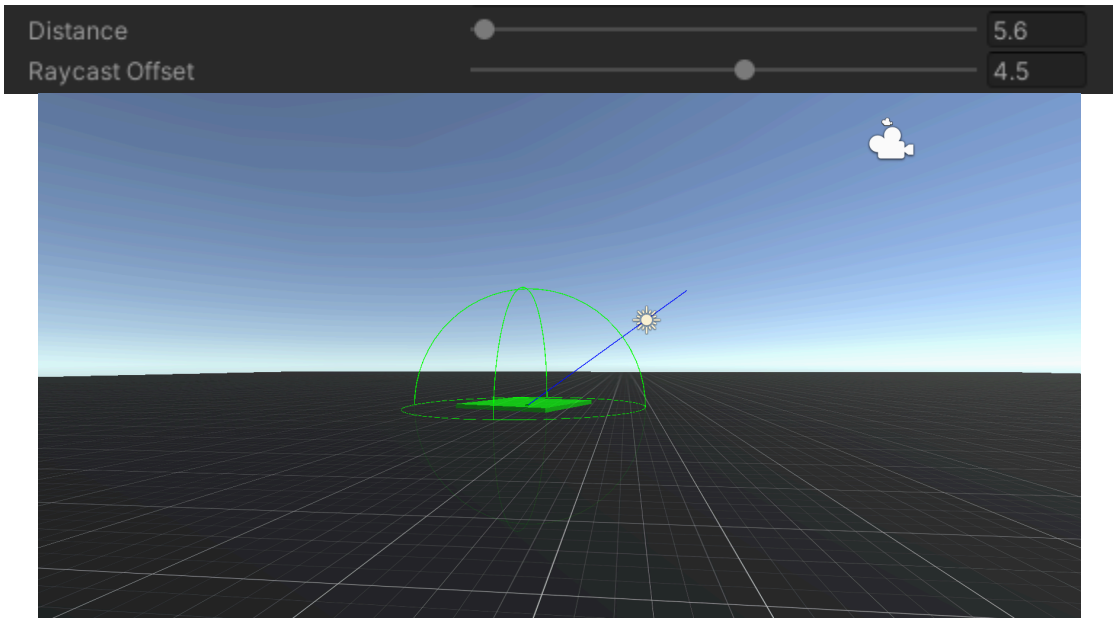
First of all the raycast has 2 different modes. The Cursor or the Camera. The Cursor mode means that the Manager is performing the raycast on the screen from the cursor's current position. It is used for those projects when the cursor should be used for building like drag and dropping a building element into the scene. The Camera is used typically in those projects when the camera position is changing when you move your cursor like FPS or a lot of TPS projects. In Camera mode the center of the screen is used for the raycast.



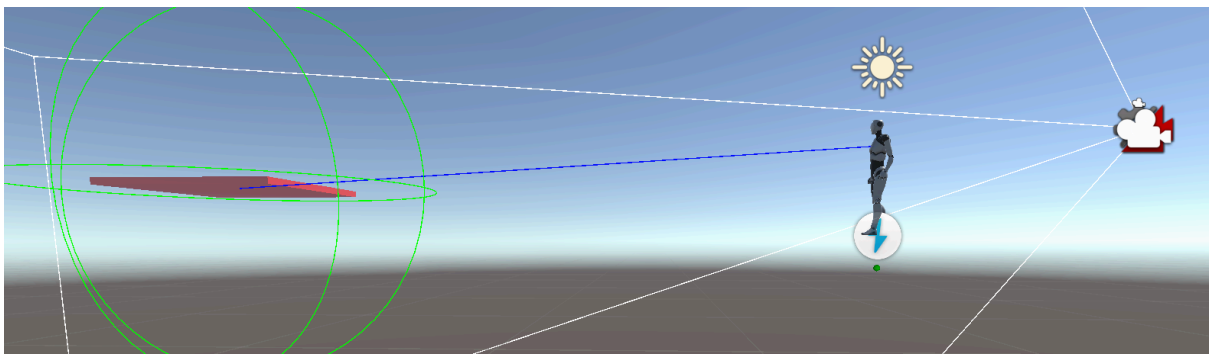
The following properties should be set up:

- Layers
  These layers will be hit by the raycast. Recommended to add the BuildingElemenets' layer too.
- Distance
  How far should the raycast go?
- Raycast offset
  An offset can be set to give the possibility to modify the startpoint of the raycast.
- Gizmos Drag Raycast Line
  Disable the raycast line's Gizmo
- Color (first)
  The color of the raycast line
- Gizmos Draw Raycast Hit Point
  Draw a sphere on the hitpoint of the raycast.
- Color (second)
  The color of the hitpoint Gizmo.





In this example above you can see with the camera offset the the raycast start position is changed. It is typically used to set a minimum distance between the BuildingElements and the camera. Also a good usage in TPS games is to set the raycast start point to the character instead of the camera.



Action History

Goal

The goal of this feature is to implement an action history that can be used to undo or redo the player actions like undo a wrongly placed element or undo the destruction of an element.

Behaviour

This history feature can save the placement of an element or the destruction of an element. Also it will save the creation or the destruction of the Buildings not just the elements.

The action history is adapting even if some action were undone. So let’s say there is 5 action like:

1. Action 1
2. Action 2
3. Action 3
4. Action 4
5. Action 5 (current last action)

After that the player undo 2 action:

1. Action 1
2. Action 2
3. Action 3 (current last action)
4. Action 4 (undone)
5. Action 5 (undone)

Then the player done an another action and after the the action history should be looks like the following:

1. Action 1
2. Action 2
3. Action 3
4. Action 6 (current last action)

In case of Partial Processing the Action History feature will try to replace (in case of redo a build action or in case of undo a destroy action) or remove (in case of undo a build action or in case of redo a destroy action) the elements until it is possible. These scenarios can only happen if the action was a DragBuilding or a DragDestroy action. Every action will be successful if at least one element can be built or removed.

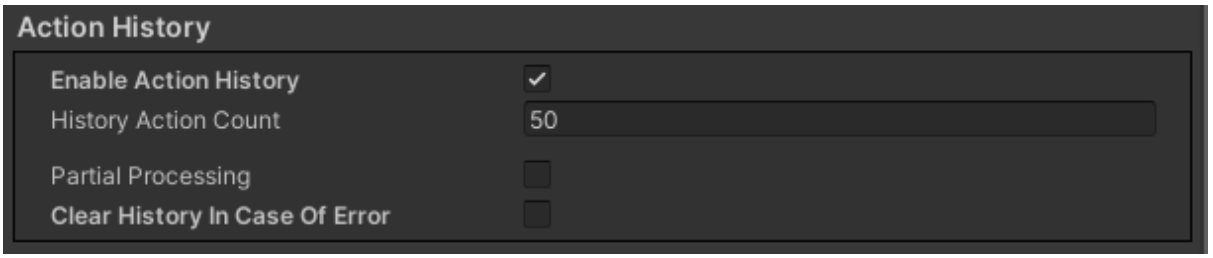
If the Partial Processing is not allowed then any error occurred then the whole action should be undone so for example the original action was a Drag Building which placed 10 elements. The player undone it successfully. After that the player tries to redo it and let's say the placement of the 5th element failed, then the first 4 elements should be removed and the redo of the action is not possible. So basically these redo and undo actions should be as atomic as possible.

An action can partially fail in many different cases, for example the BuildingManager placed 10 elements at once. But one of the element's is destroyed because of an independent event from the BuildingManager. Because of the independent action the Building Manager thinks that the 10 elements are still available and it will just realize the missing element when the build action of the 10 element will be undone. So the failures can be caused by independent events from the BuildingManager.

The Action History will trigger some function on the trackers such as an element will be destroyed/placed. An action can be failed too when an action is denied by one of the trackers.

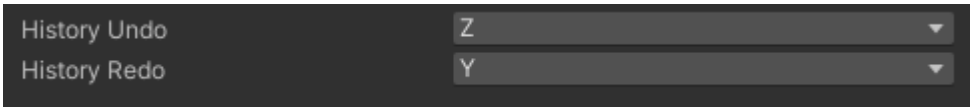
Note that this feature is the BuildingManager's feature so it will not know anything about other BuildingManager's actions.

Settings



The settings of the feature are already explained so here will be just a short introduction. First of all the feature can be enabled or disabled.
The number of the saved action can be set on the UI. If more action happens than the set number then the oldest action is removed.

The following 2 legacy key can be set to the feature:



Note that in a lot of cases the Action History will use the BuildingElementList that is provided to the BuildingManager to instantiate the elements so ensure that the list is containing every element.

Note that the actions can be cleared any time with the ClearHistory function on the IBuildingManagerExternalInterface interface.

Requirements

The Action history feature's requirements are the followings:

- **If an element is replaced because of an undo of a destroy action or a redo of a build action the followings should be guaranteed:**
    - The element should be the same prefab as the original one.
    - The position should be the same as the original's position.
    - The rotation should be the same as the original's rotation.
    - The parent building should be the same as the original's parent.
        - The only exception is when the building is destroyed and the placement of the new element should recreate the building too.
    - The InstanceGuid should be the same as the original's InstanceGuid.
    - If the original was a prebuilt element the new one should be prebuilt too.
    - The replacement of the new element can cause the destruction of other elements in the following scenarios:
        - The placement of an element can override an element if the original action was an override build action.
        - The placement of the new element can be done in place of a prebuilt element if the action was originally placed into the prebuilt element.
- **If an element is removed because of an undo of a build action the followings should be guaranteed:**
    - If the building has no more elements then the building should be destroyed but its data should be saved to recreate it later.
    - If the build action was overridden an element or it was built into a prebuilt element's position then the undo should replace the original element with the followings should be the same as the original's:
        - Same prefab should be used
        - Instance Guid
        - Position
        - Rotation
        - PreBuilt state
        - Parent Building (Recreate it if is required)
- **If an element is removed because of a redo of a destroy action the followings should be guaranteed:**
    - If the building has no more elements then the building should be destroyed but its data should be saved to recreate it later.
- **Creation of a building**
    - The building should be in the same position as the original one.
    - The building should have the same rotation as the original one.
    - The building should have the same Maximum Element Count as the original one.
    - The cache usage of the new building should match with the original one so if it was disabled then on the new one it should be disabled too.
    - In case of AdvancedGridBuilding
        - The building should has the same Position Modifier as the original one
- **Drag Building/Destroy**
    - The action history should be able to handle the DragBuilding and DragDestroy scenarios as well.